Program 2

Step 1: Create index.js in directory /src

```
1rv24mc024_chandan_athreya_s@JARVIS:~/Devops-LabSessions/prog2$ cat src/index.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('hello from multistage docker');
});

app.listen(PORT, () => {
  console.log(`server running on port ${PORT}`);
});
```

Step 2: Run npm init -y

```
1rv24mc024_chandan_athreya_s@JARVIS:~/Devops-LabSessions/prog2$ npm init -y
Wrote to /home/chandan-athreya-s/Devops-LabSessions/prog2/package.json:

{
  "name": "prog2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node dist/index.js",
    "build": "mkdir -p dist && cp -r src/* dist/"
  },
  "keywords": [],
  "author": "Chandan",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  },
  "devDependencies": {},
  "type": "commonjs"
}
```

Step 3: Create a Dockerfile

```
1rv24mc024_chandan_athreya_s@JARVIS:~/Devops-LabSessions/prog2$ cat Dockerfile
#Stage1: Build Stage
FROM node:20-alpine AS builder

#Set working Directory
WORKDIR /app

#copy package files and install dependencies
COPY package.json package-lock.json ./
RUN npm install

#Copy Application Source Code
COPY . .

#Build the Application
RUN npm run build

#Stage2: Production Stage
FROM node:20-alpine

#Set Working Directory
WORKDIR /app

#Copy only necessary files from build stage
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

#Expose the Application port
EXPOSE 3000

#start the application
CMD ["node", "dist/index.js"]
```

Step 4: Create a Docker image using Dockerfile

```
1rv24mc024_chandan_athreya_s@JARVIS:~/Devops-LabSessions/prog2$ docker build -t program-2 .
[+] Building 7.9s (15/15) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                            0.0s
 => => transferring dockerfile: 713B                                                            0.0s
 => [internal] load metadata for docker.io/library/node:20-alpine                               2.4s
 => [internal] load .dockerignore                                                               0.0s
 => => transferring context: 2B                                                                 0.0s
 => [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435  0.0s
 => [internal] load build context                                                               0.1s
 => => transferring context: 99.70kB                                                            0.0s
 => CACHED [builder 2/6] WORKDIR /app                                                           0.0s
 => [builder 3/6] COPY package.json package-lock.json ./                                        0.0s
 => [builder 4/6] RUN npm install                                                               3.7s
 => [builder 5/6] COPY . .                                                                      0.5s
 => [builder 6/6] RUN npm run build                                                             0.5s
 => [stage-1 3/6] COPY --from=builder /app/package.json ./                                      0.0s
 => [stage-1 4/6] COPY --from=builder /app/package-lock.json ./                                 0.0s
 => [stage-1 5/6] COPY --from=builder /app/dist ./dist                                          0.0s
 => [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules                          0.1s
 => exporting to image                                                                          0.2s
 => => exporting layers                                                                         0.1s
 => => writing image sha256:ce8d5d0beae0e731fd82b0a6d37b4b853cabb00e0c285bc3623dd0a24d99c587    0.0s
 => => naming to docker.io/library/program-2                                                    0.0s
```

Step 5: Create a Docker container using the Docker Image

```
1rv24mc024_chandan_athreya_s@JARVIS:~/Devops-LabSessions/prog2$ docker run -p 3000:3000 program-3
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.18.0.2:5000
Press CTRL+C to quit
```